

Java Refresher: Object-Oriented Programming (OOP)

Welcome to this quick refresher on Object-Oriented Programming (OOP) in Java. Understanding OOP principles is fundamental for building robust and scalable applications, especially when working with frameworks like Spring Boot.

1. What is OOP?

OOP is a programming paradigm based on the concept of 'objects', which can contain data and code: data in the form of fields (attributes or properties), and code in the form of procedures (methods). It's designed to make software development more intuitive, modular, and maintainable.

2. Four Pillars of OOP

Let's quickly review the four core principles of OOP:

- **Encapsulation:** Bundling the data (attributes) and methods that operate on the data into a single unit or class. It also involves restricting direct access to some of an object's components, which means direct modification of data is prevented. We typically use access modifiers (`private`, `public`, `protected`, default) for this.
- **Inheritance:** A mechanism where one class (subclass/child class) acquires the properties and behaviors of another class (superclass/parent class). This promotes code reusability. In Java, a class can extend only one other class (single inheritance), but can implement multiple interfaces.
- **Polymorphism:** The ability of an object to take on many forms. The most common use of polymorphism in OOP is when a parent class reference is used to refer to a child class object. It allows you to define one interface or method and have multiple implementations. In Java, this is achieved through method overloading (compile-time polymorphism) and method overriding (runtime polymorphism).
- **Abstraction:** Hiding the complex implementation details and showing only the necessary features of an object. This can be achieved in Java using abstract classes and interfaces. It helps in managing complexity by focusing on 'what' an object does rather than 'how' it does it.

3. Key Concepts & Keywords in Java OOP

- **Class:** A blueprint or template from which objects are created. It defines a data type by combining data and methods into a single unit. Example:

```
public class Car {    String make;    String model;    int year;    public void start() {        System.out.println("Car started.");    }    pub
```

- **Object:** An instance of a class. When a class is defined, no memory is allocated, but when an object is created, memory is allocated. Example:

```
Car myCar = new Car();myCar.make = "Toyota";myCar.model = "Camry";myCar.year = 2023;myCar.start();
```

- **Constructor:** A special method used to initialize objects. It has the same name as the class and does not have a return type. Example:

```
public class Person {    String name;    int age;    public Person(String name, int age) { // Constructor        this.name = name;        this.a
```

- **Methods:** Blocks of code that perform actions. They define the behavior of an object.
- **Attributes (Fields):** Variables defined within a class but outside any method, constructor, or block. They represent the state of an object.
- **this keyword:** Refers to the current object. It is used to distinguish instance variables from local variables if they have the same name.
- **super keyword:** Used to refer to the immediate parent class object. It can be used to call parent class constructors and methods.
- **static keyword:** Can be applied to variables, methods, blocks, and nested classes. A static member belongs to the class itself rather than to any specific instance of the class.
- **final keyword:** Can be applied to variables (makes them constants), methods (prevents overriding), and classes (prevents inheritance).
- **Interfaces:** A completely abstract class that is used to group related methods with empty bodies. It specifies a contract that implementing classes must adhere to. Example:

```
public interface Drivable {    void accelerate();    void brake();}public class Sedan implements Drivable {    @Override    public void accelera
```

- **Abstract Classes:** Classes that cannot be instantiated and may contain abstract methods (methods without a body). Subclasses must provide implementations for all abstract methods. Example:

```
public abstract class Shape {    public abstract double calculateArea(); // Abstract method    public void display() {        System.out.println
```

These OOP concepts are heavily utilized in Spring Boot development, from defining components as classes and interfaces to leveraging inheritance and polymorphism for flexible and extensible application architectures. A strong grasp of these fundamentals will make learning Spring Boot much easier.